Taller 7 — El personaje de Pixel Dungeon Lite cobra vida

Sesión 7 · Corte 2 · Grupo 10 · **Individual, sobre el repositorio del equipo**

Entrega: al terminar la clase, un documento por estudiante (PDF o enlace) en su fila del Excel del corte, con su **usuario de GitHub** y la **URL del repositorio**. El código se revisa en sus commits.

> **Qué van a hacer.** En la demo, el personaje aprendió a caminar en 3D: se mueve relativo a la cámara, mira hacia donde camina y cambia entre *idle* y *walk*. Hoy empieza a existir **Pixel Dungeon Lite**, el juego de su Documento de concepto. Cada integrante toma **una tarea** (el movimiento del jugador, una acción con tiempo o un personaje que se mueve solo), la construye en el **repositorio del equipo**, en **su propio archivo**, y llena con ella un **vacío** del documento: un número que faltaba, una velocidad, un tiempo o una regla. Cada tarea tiene tres niveles; cada nivel termina en un **commit** con su usuario.
>
> **Pixel Dungeon Lite en una frase** (de su Documento de concepto): "Es Soul Knight con salas fijas y un jefe final, para jugadores que buscan retos cortos."
>
> **Es individual, pero sobre un proyecto compartido.** Cada integrante hace una tarea distinta y la escribe en **su propio archivo**. Nadie edita el archivo de otro. El documento es de cada uno; el código es de quien lo comete.
>
> **Cómo se trabaja.** Cada nivel tiene las mismas partes, en este orden. **Antes de programar:** preguntas que se responden en papel, con números; copien las respuestas al documento **antes** de escribir código. **Paso a paso:** qué hacer en el editor y en el código; hagan un paso, corran el juego (F5) y miren si pasa lo que dice **Qué deben ver**. **Esqueleto:** el código con huecos _____ que escriben ustedes. **Pistas:** en orden y solo si se traban; ninguna es la solución. **Errores típicos:** si algo raro pasa, busquen ahí el síntoma. Al final de cada nivel, **Commit**: el mensaje exacto que lleva, y **Qué va al documento**.
>
> Una predicción equivocada pero bien explicada vale más que una correcta sin explicación.

Reparto del trabajo

Integrante	Tarea	Archivos que crea o edita	Commits esperados
Barriga Castro Isaac Leonardo	A — El héroe se mueve	*personaje.gd*	`T7 N1 - Tarea A - correr y la diagonal` · `T7 N2 - Tarea A - arranque y frenado` · `T7 N3 - Tarea A - estados con su número`
Gómez Valbuena Diego Alejandro	B — La espada	*<accion>.gd* (y su nodo en *main.tscn*)	`T7 N1 - Tarea B - el nodo de la accion` · `T7 N2 - Tarea B - estados con tiempo` · `T7 N3 - Tarea B - enchufes y regla`
Mejía Poveda Bryan Smith	C — El perseguidor	*<enemigo>.tscn* y *<enemigo>.gd*	`T7 N1 - Tarea C - escena del enemigo` · `T7 N2 - Tarea C - sigue al jugador` · `T7 N3 - Tarea C - patrulla y persecucion`

Nadie edita el archivo de otro. Si quieren cambiar el reparto, pueden, pero todos de acuerdo y escribiéndolo en sus documentos. **Quién crea el repositorio:** Barriga Castro Isaac Leonardo (el de la tarea A).

Duración

	Grupos de 3 horas	Grupos de 2 horas
Tiempo para el taller	105 minutos	45 minutos
Hasta dónde se espera llegar	Nivel 3 de su tarea	Nivel 2 de su tarea (el nivel 3 da acceso al 5.0)

Tiempo sugerido en 3 horas: la base 10 min, nivel 1 20, nivel 2 35, nivel 3 30 y documento 10. En 2 horas: la base 10, nivel 1 15, nivel 2 15 y documento 5.

Parte 1 — La base del equipo (todos, 10 minutos)

Qué trae la plantilla 3D al abrirla. La escena *main.tscn* tiene: **WorldEnvironment** (el cielo), **DirectionalLight3D** (el sol, con sombras), **Suelo** de 30 × 30 m, **Cajas** (tres cajas naranjas de 1 m; la Caja1 está en (4, 0.5, −4), la Caja2 en (−5, 0.5, −2) y la Caja3 en (3, 0.5, 3)), **Camera3D** (fija, en (0, 5, 7) e inclinada 35° hacia abajo) y **Personaje**. El Personaje es un CharacterBody3D con **CollisionShape3D** (cápsula de radio 0.35 y altura 1.75) y **Modelo** (*character-male-a.glb*, con Rotation Y = 180 y Scale 2.6). En el Input Map solo están *move_left*, *move_right*, *move_up* y *move_down* (WASD y flechas): **no** hay *correr* ni *saltar*. *personaje.gd* es el de la demo (con `speed = 3.0` y la dirección con `camara.global_basis`), más los tres enchufes y el bloque de pruebas. En la carpeta *personaje/*

hay otros tres modelos de Kenney. Con F5, el personaje camina a 3 m/s con *idle* y *walk*.

El repositorio (lo hace **Barriga Castro Isaac Leonardo**; los demás miran):

1. Descarguen **Inicio del juego 3D.zip** de Teams, descomprímanlo y cambien el nombre de la carpeta por el del juego (sin tildes ni espacios raros). Impórtenla en Godot (**Import** → *project.godot* → **Import & Edit**) y esperen a que termine de importar el personaje.
2. En **GitHub Desktop**: *File* → *Add local repository* → la carpeta del juego → *create a repository* → **Create**. Después **Publish repository**, **sin** marcar *Keep this code private*: el repositorio es **público**.
3. Copien la URL (*Repository* → *View on GitHub*) y péguenla en la hoja **Repositorio del juego** del Excel del corte, en la fila de su equipo. Una sola vez por equipo.
4. Los demás: GitHub Desktop → *File* → *Clone repository* → la URL. Cada uno abre su copia en Godot.

La cuenta. Cada uno entra con **su** usuario en GitHub Desktop (*File* → *Options* → *Accounts* → *Sign in*), **en su propio PC**. Un commit hecho con la cuenta de otro cuenta para el otro: así se califica.

Un commit por nivel. Al terminar cada nivel: GitHub Desktop → marcar sus archivos → escribir el mensaje **exacto** que dice la guía (por ejemplo `T7 N1 - Tarea A - correr y la diagonal`) → **Commit** → **Push origin**. Antes de empezar un nivel, **Fetch origin** y **Pull**, para traer lo de los demás. Como cada uno edita archivos distintos, no hay conflictos; si GitHub Desktop avisa uno, es que dos personas tocaron el mismo archivo: avisen al profesor.

Plan B. Si el repositorio no sale en 10 minutos, no se queden esperando: cada uno trabaja en su propia copia de la plantilla y al final de la clase **Barriga Castro Isaac Leonardo** junta los archivos y hace los commits a nombre de cada uno, o cada uno entrega un .zip. Vale menos: sin commit propio, el nivel se califica solo por el documento.

> **Los tres enchufes.** Arriba de *personaje.gd* hay tres variables que **otros scripts** pueden cambiar. Así, la tarea B se escribe en su propio archivo y no toca el de la tarea A:
>
> - `multiplicador`: multiplica la rapidez de las teclas. 1 es normal, 0.5 es la mitad y 0 deja al personaje quieto aunque se presionen las teclas.
> - `empuje`: una velocidad que se **suma**, en m/s. Sirve para un dash, un retroceso o un empujón.
> - `accion`: el nombre de una animación. Mientras no esté vacío, el personaje hace esa animación en vez de *idle* o *walk*. Las de Kenney son *attack-melee-right*, *attack-kick-right*, *crouch*, *die*, *interact-right*, *pick-up*, *holding-right-shoot*, *drive*, *sit*, *emote-yes* y *emote-no*, además de *idle*, *walk*, *sprint*, *jump* y *fall*.
>
> La tarea A **no** borra los enchufes: las líneas de `velocity` los tienen que seguir usando.
>
> **La prueba automática de la plantilla.** Sirve para cualquier juego y la usan las tres tareas. Seleccionen **Personaje** → Inspector → **Prueba**:
>
> - *Avanzar 60 cuadros*: mantiene W (en 2.5D, D) durante 60 cuadros.
> - *Avanzar 60 y soltar*: mantiene W 60 cuadros, la suelta y sigue escribiendo hasta el cuadro 90. Sirve para ver cómo frena.
> - *Avanzar y la acción desde el cuadro 30*: mantiene W y, desde el cuadro 30, también la acción que escriban en **Accion Prueba** (por ejemplo, *correr*).
> - *Mantener la acción 2 s*: mantiene solo la acción de **Accion Prueba** durante 120 cuadros, sin moverse.
> - *Avanzar con la acción 60 cuadros*: W y la acción juntas desde el cuadro 1.
> - *Diagonal W+D 60 cuadros*: W y D juntas (solo tiene sentido en 3D).
>
> En cada línea escribe la posición, la rapidez en el suelo, la altura `y`, la animación y la variable `estado` del Personaje y de sus nodos hijos (si la tienen). Además, cada vez que un `estado` cambia escribe una línea con el cuadro exacto, y al final, si saltó, la altura máxima. Se toca F5 sin tocar el teclado. Al terminar, dejen **Prueba** en *Ninguna*.

La tabla de números del juego. Cada uno la lleva en su documento, con una fila por número que use su tarea:

Qué es	Número	¿Ya estaba en el documento o lo propuso?	Por qué ese número	Lo que midió la prueba
Rapidez al caminar (ejemplo)	2.5 m/s	Propuesto (vacío del documento)	Cruzar la sala de 10 m toma 4 s	Avanzar 60 cuadros: z = −2.21, rapidez 2.5

Las filas que ya estaban en el documento se copian **sin cambiarlas**. Si al probarlas no funcionan en el juego, se anota en la última columna y se le cuenta al equipo; el cambio se decide en Desarrollo conceptual.

Tarea A — Barriga Castro Isaac Leonardo: El héroe se mueve

Sigue la **receta A (el movimiento del jugador)**. Tiene tres niveles: cada uno termina en un commit.

En su juego. El héroe camina por la sala con WASD.

Lo que ya dice su documento. WASD, clic izquierdo espada y clic derecho flecha. Invulnerable **1.5 s** tras el daño. Cópienlo tal cual en la tabla.

El vacío que llena. La rapidez del héroe; la escala de los píxeles. Propongan el número, escriban por qué y compruébenlo con la prueba.

Cada nivel se hace **dos veces**: primero con los **números del ejemplo** de la receta, que no son de ningún equipo (si la prueba da lo mismo que *Qué deben ver*, el código está bien), y después con los **números de su juego**, que van a la tabla.

Nivel 1 — Correr, y una diagonal que no es diagonal

Qué ya saben. En la demo, `Input.get_vector(...)` dio la entrada como un Vector2 (x a los lados, y adelante y atrás) y se pasó a 3D con `Vector3(input_dir.x, 0.0, input_dir.y)`. Multiplicarla por `camara.global_basis` la convierte al punto de vista de la cámara; `direction.y = 0.0` la aplana contra el suelo, y `normalized()` le deja largo 1 para que la diagonal no vaya más rápido. La rapidez es `speed`. Así viene *personaje.gd* en la plantilla.

El ejemplo: caminar a **3 m/s** (lo que trae la plantilla) y correr a **6 m/s** con Shift.

Antes de programar

1. Con la prueba *Avanzar 60 cuadros*, el personaje camina 60 cuadros a 3 m/s desde (0, 0, 0), y la cámara mira hacia −Z. ¿En qué posición termina? Si corre a 6 m/s, ¿dónde termina?
2. La prueba *Diagonal W+D 60 cuadros* mantiene W y D: la entrada es `Vector3(1, 0, −1)`. La cámara está inclinada 35° hacia abajo, y por eso `camara.global_basis * Vector3(1, 0, −1)` da **(1, −0.57, −0.82)**: la parte de W se reparte entre "abajo" y "adelante". Después de `direction.y = 0.0` y `normalized()`, ¿qué dirección queda? ¿Dónde termina el personaje después de 60 cuadros a 3 m/s? ¿Es una diagonal de 45°?
3. Para arreglarlo, se gira la entrada solo con el **giro horizontal** de la cámara: `entrada.rotated(Vector3.UP, camara.global_rotation.y)`. En la plantilla ese giro es 0. ¿Dónde termina ahora la diagonal? Y si se les olvida `normalized()`, ¿a qué rapidez va en diagonal?

Guía para la 1. Distancia = rapidez × tiempo, y 60 cuadros son 1 segundo. Con otros números: a 2.5 m/s durante 90 cuadros (1.5 s) se recorren 3.75 m. Como W va hacia −Z, la coordenada que cambia es z, y se vuelve negativa.

Guía para la 2. Con otros números: si después de aplanar la dirección queda (1, 0, −0.5), su largo es $\sqrt{(1^2 + 0.5^2)}$ = 1.118; normalizada queda (1 ÷ 1.118, 0, −0.5 ÷ 1.118) = (0.894, 0, −0.447). A 2 m/s durante 1 s, termina en (1.79, 0, −0.89). En una diagonal de 45°, x y z tienen el mismo tamaño.

Guía para la 3. `rotated(Vector3.UP, 0)` no cambia nada: la entrada queda (1, 0, −1), de largo √2 = 1.414. Normalizada, cada componente vale 1 ÷ 1.414 = 0.707. Sin normalizar, la rapidez queda multiplicada por 1.414.

Con otros números: a 5 m/s, sin normalizar, en diagonal iría a 7.07 m/s.

Paso a paso

1. **Antes de tocar nada**, corran la prueba *Diagonal W+D 60 cuadros* (Personaje → Inspector → **Prueba**) y anoten dónde termina. Después *Avanzar con la acción 60 cuadros* con **Accion Prueba** = *correr*: el Output avisa que falta la acción y el personaje camina a 3. Anótenlo.
2. **La acción correr.** *Project* → *Project Settings* → pestaña **Input Map**. En *Add New Action* escriban **correr** y presionen **+ Add**. En la fila de *correr*, clic en el **+** de la derecha → presionen la tecla **Shift** → **OK**. Cierren la ventana.
3. En *personaje.gd*, cambien `@export var speed = 3.0` por las dos rapideces del esqueleto: `speed_caminar` y `speed_correr`.
4. En `_physics_process()`, **borren** las líneas de `direction` (la de `camara.global_basis`, la de `direction.y = 0.0` y la de `normalized()`) y escriban las dos líneas nuevas con `rotated` (esqueleto).
5. Debajo, elijan la rapidez: si se mantiene **correr**, `speed_correr`; si no, `speed_caminar`. Usen esa `speed` en las dos líneas de `velocity`. **No borren** `* multiplicador` ni `+ empuje`: son los enchufes que usa la tarea B.
6. Cambien la animación: si se mueve y corre, **sprint**; si se mueve y no corre, **walk**; si está quieto, **idle**.
7. Corran *Avanzar 60 cuadros*, *Avanzar con la acción 60 cuadros* (con *correr*) y *Diagonal W+D 60 cuadros*, en ese orden, y anoten los tres resultados.
8. Pruébenlo a mano: giren la cámara (**Camera3D** → Transform → **Rotation Y = 90**), F5, y comprueben que W sigue llevando al personaje "hacia el fondo de la pantalla". Vuelvan la cámara a 0.

Esqueleto.

```
@export var camara: Camera3D
@export var speed_caminar = ____
@export var speed_correr = ____
@onready var anim = $Modelo/AnimationPlayer


func _physics_process(delta):
    velocity += get_gravity() * delta

    var input_dir = Input.get_vector("move_left", "move_right",
        "move_up", "move_down")
    var entrada = Vector3(input_dir.x, 0.0, input_dir.y)
    # BORREN las 3 líneas de antes:
    #   var direction = camara.global_basis * entrada
    #   direction.y = 0.0
    #   direction = direction.normalized()
    var giro_cam = camara.global_rotation.____
    var direction = entrada.rotated(Vector3.UP, giro_cam).____()

    var corre = Input.is_action_pressed("____")
    var speed = ____ if corre else ____
    velocity.x = direction.x * speed * multiplicador + empuje.x
    velocity.z = direction.z * speed * multiplicador + empuje.z

    if direction.length() > 0:
        basis = Basis.looking_at(direction)
    if accion != "":
        anim.play(accion)
    elif direction.length() > 0 and corre:
        anim.play("____")
    elif direction.length() > 0:
        anim.play("walk")
    else:
        anim.play("idle")

    move_and_slide()
    _prueba()
```

Qué deben ver.

Prueba	Plantilla (antes)	Con el nivel 1
Avanzar 60 cuadros	(0, 0, −3) · rapidez 3 · walk	(0, 0, −3) · rapidez 3 · walk
Avanzar con la acción 60 cuadros (*correr*)	"falta la acción 'correr'" · (0, 0, −3) · 3	**(0, 0, −6) · rapidez 6 · sprint**
Diagonal W+D 60 cuadros	**(2.32, 0, −1.9)** · rapidez 3	**(2.12, 0, −2.12)** · rapidez 3

En la plantilla, la diagonal queda torcida: avanza 2.32 hacia la derecha y solo 1.9 hacia adelante. Con `rotated`, los dos lados miden 2.12: es una diagonal de verdad. Con la cámara girada 90°, W lleva al personaje hacia −X: sigue siendo "hacia el fondo de la pantalla".

Pistas

- **Pista 1:** el giro horizontal de un nodo 3D es su rotación alrededor del eje Y. En `camara.global_rotation` está en radianes, en sus tres componentes x, y, z.
- **Pista 2:** `condición ? a : b` no existe en GDScript; se escribe `a if condición else b`. Lo que va en la condición es una pregunta al Input que devuelve verdadero mientras la tecla está presionada.
- **Pista 3:** si la diagonal sigue dando 2.32 y −1.9, todavía queda en algún lado la multiplicación por `camara.global_basis`.

Errores típicos

- *"The InputMap action "correr" doesn't exist":* la acción no se creó, o se escribió distinto (Correr, correr con un espacio).
- *Corre aunque no presione Shift:* quedaron al revés los dos lados del `if ... else` de `speed`.
- *Corre más rápido pero sigue con la animación de caminar:* falta el `and corre`, o el nombre está mal escrito (las animaciones de Kenney van en minúscula: *sprint*, no *Sprint*).
- *La diagonal da 4.24 de rapidez:* falta `normalized()`.
- *"Invalid access to property or key 'global_rotation' on a base object of type 'Nil'":* la propiedad **Camara** del Personaje está vacía y no hay otra cámara; en el Inspector del Personaje arrastren **Camera3D** hasta **Camara**.
- *El dash de la tarea B deja de funcionar:* borraron `+ empuje.x` o `+ empuje.z`.

Con los números de su juego (nivel 1). Pongan en el Inspector la rapidez de caminar y la de correr de su juego (si su juego no corre, las dos iguales; si la tecla sirve para ir más **lento**, `speed_correr` menor que `speed_caminar`) y repitan *Avanzar 60 cuadros* y *Avanzar con la acción 60 cuadros*. Los números están en el recuadro *Con los números de su juego (nivel 2)*, más abajo: si alguno falta en el documento, propónganlo.

Commit. `T7 N1 - Tarea A - correr y la diagonal`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

Nivel 2 — El movimiento del jugador, con arranque y frenado

Qué ya saben. Del nivel 1: girar la entrada con `rotated(Vector3.UP, ...)` y elegir la rapidez con `a if condición else b`. Lo nuevo es `move_toward(a, b, paso)`: acerca el valor `a` al valor `b`, pero sin avanzar más de `paso` en cada llamada. Por ejemplo, `move_toward(0, 10, 3)` da 3, y llamado otra vez desde 3 da 6. Funciona igual con un Vector3. La variable `avance` guarda la parte de la velocidad que viene de las teclas, para que crezca de a poco en cada cuadro.

El ejemplo: caminar a 2.5 m/s, correr a 5 m/s y `tiempo_arranque = 0.25`. Es decir, tarda 0.25 s en llegar a la rapidez que toca y otros 0.25 s en frenar del todo. En 2.5D, el ejemplo es: rapidez 5 m/s, `tiempo_arranque = 0.2`, gravedad 20 m/s² y un salto de 1.5 m de alto.

Antes de programar

1. El paso de cada cuadro es `speed ÷ tiempo_arranque × delta`. Caminando, ¿cuánto sube la rapidez en cada cuadro? ¿En qué cuadro llega a 2.5?
2. Sin arranque, en 60 cuadros avanzaría 2.5 m. Con arranque, ¿cuánto avanza? (Mientras arranca, va en promedio a la mitad de la rapidez.)
3. Al soltar W cuando va a 2.5 m/s, ¿en cuántos cuadros se detiene? ¿Cuántos metros recorre mientras frena?

Guía para la 1. Con otros números: para llegar a 4 m/s en 0.5 s, la rapidez sube 4 ÷ 0.5 = 8 m/s cada segundo. Por cuadro, 8 ÷ 60 = 0.133, así que llega a 4 en el cuadro 30.

Guía para la 2. Con los mismos otros números (4 m/s y 0.5 s): en el primer medio segundo recorre 4 × 0.5 ÷ 2 = 1 m, no 2. En 1 s recorre 1 m + 4 × 0.5 = 3 m, en vez de 4.

Guía para la 3. Frenar es lo mismo al revés: el paso es igual y la cuenta del promedio también.

Paso a paso

1. Sigan en *personaje.gd*, el del nivel 1.
2. **La acción.** La acción **correr** ya existe desde el nivel 1. Si en su juego esa tecla sirve para ir más **lento** (sigilo, caminar con Ctrl), es la misma receta con `speed_correr` menor que `speed_caminar`. En 2.5D no hay acción de correr: sáltense este paso.
3. **Las variables.** Agreguen `tiempo_arranque` debajo de las dos rapideces del nivel 1, y `var avance` debajo de los enchufes.
4. **La dirección.** Ya es la del nivel 1, con `rotated`: no se toca. En 2.5D no hay `direction`: la entrada es `lado`.
5. **La velocidad.** Cambien las dos líneas de `velocity` por las del esqueleto: el objetivo, el paso, el `move_toward` y las dos líneas de `velocity` con el `empuje`.
6. **La animación.** Cambien la condición de *walk* para que mire `avance`, y agreguen *sprint* si su juego corre.
7. En el Inspector del Personaje, pongan los números del ejemplo y corran las tres pruebas: *Avanzar 60*

cuadros; *Avanzar y la acción desde el cuadro 30* con **Accion Prueba** = *correr*; y *Avanzar 60 y soltar*. Comparen con **Qué deben ver**.

8. **Con sus números.** Pongan en el Inspector los números de su juego (recuadro *Con los números de su juego*, abajo). Si el número falta en el documento, propónganlo y anoten en la tabla por qué ese. Corran las mismas pruebas y anoten los resultados en la tabla.

Esqueleto 3D.

```
@export var camara: Camera3D
@export var speed_caminar = ____
@export var speed_correr = ____
@export var tiempo_arranque = ____
@onready var anim = $Modelo/AnimationPlayer

var multiplicador = 1.0
var empuje = Vector3.ZERO
var accion = ""
var avance = Vector3.ZERO   # la parte de la velocidad que dan las teclas

    # (dentro de _physics_process, después de la línea de entrada)
    var giro_cam = camara.global_rotation.y
    var direction = entrada.rotated(Vector3.UP, giro_cam).normalized()

    var corre = Input.is_action_pressed("correr")
    var speed = speed_correr if corre else speed_caminar
    var objetivo = direction * speed * ____
    var paso = speed / ____ * delta
    avance = avance.move_toward(____, paso)
    velocity.x = avance.x + empuje.x
    velocity.z = avance.z + ____

    if direction.length() > 0:
        basis = Basis.looking_at(direction)
    if accion != "":
        anim.play(accion)
    elif avance.length() > speed_caminar + 0.1:
        anim.play("____")
    elif avance.length() > 0.1:
        anim.play("walk")
    else:
        anim.play("idle")
```

Esqueleto 2.5D (cambia el salto y la velocidad; el resto queda como en la plantilla):

```
@export var speed = ____
@export var tiempo_arranque = ____
@export var gravedad = ____
@export var altura_salto = ____

var avance = 0.0   # la parte de la velocidad que dan las teclas

    # (dentro de _physics_process)
    if not is_on_floor():
        velocity.y -= gravedad * delta
    elif Input.is_action_just_pressed("saltar"):
        velocity.y = sqrt(2 * ____ * ____)

    var lado = Input.get_axis("move_left", "move_right")
    var paso = speed / tiempo_arranque * delta
    avance = move_toward(avance, lado * speed * multiplicador, ____)
    velocity.x = avance + empuje.x
```

En 2.5D el salto por altura ya quedó en el nivel 1; lo nuevo son `tiempo_arranque`, `avance` y el `move_toward`. En la animación, cambien `elif lado != 0:` por `elif absf(avance) > 0.1:`.

Qué deben ver (con los números del ejemplo, en 3D):

Prueba	Con el nivel 1	Con el nivel 2
Avanzar 60 cuadros	rapidez 3 desde el cuadro 1 · final (0, 0, −3)	cuadro 1: 0.17 · cuadro 10: 1.67 · cuadro 15: **2.5** · final **(0, 0, −2.21)**
Avanzar y la acción desde el cuadro 30 (*correr*)	cuadro 31: **6.0** de golpe · cuadro 60: (0, 0, −4.5)	cuadro 31: 2.83 y *sprint* · cuadro 35: 4.17 · cuadro 40: **5.0** · cuadro 90: (0, 0, −5.82)
Avanzar 60 y soltar	cuadro 61: rapidez 0 (frena de golpe)	cuadro 61: 2.33 · cuadro 65: 1.67 · cuadro 70: 0.83 · cuadro 80: **0**, quieto en z = −2.5

Arranca en 15 cuadros (0.25 s) y frena en otros 15. Mientras arranca pierde unos 0.3 m: llega a −2.21 en vez de −2.5. En 2.5D, con el ejemplo: *Avanzar 60 cuadros* da 4.17 en el cuadro 10, 5.0 desde el cuadro 12 y termina en **(4.54, 0, 0)**. *Mantener la acción 2 s* con **Accion Prueba** = *saltar* termina con "altura máxima **1.57** en el cuadro **24**": la cuenta da 1.5 m con v = √60 = 7.75 m/s, y el juego, que suma cuadro por cuadro, llega un poco más alto.

Pistas

- **Pista 1:** `move_toward` recibe tres cosas: desde dónde (lo que lleva, `avance`), hacia dónde (`objetivo`) y cuánto como máximo en este cuadro (`paso`).
- **Pista 2:** el objetivo es la dirección por la rapidez, y por el enchufe que la puede bajar a la mitad o a 0.

- **Pista 3:** si la rapidez se queda en 0.17 y no sube, `avance` se está creando de nuevo en cada cuadro: tiene que estar declarada arriba, fuera de la función.

Errores típicos

- *Llega de golpe a la rapidez, en el cuadro 1:* las líneas de `velocity` usan `objetivo` en vez de `avance`, o falta `* delta` en el paso.
- *La rapidez se queda en 0.17:* `var avance` está dentro de `_physics_process()` (ver la pista 3).
- *El dash o el empujón de la tarea B ya no hacen nada:* borraron el `+ empuje.x` o el `+ empuje.z`.
- *"The InputMap action "correr" doesn't exist":* falta la acción, o tiene otro nombre. Si su juego no corre, dejen `speed_correr` igual a `speed_caminar`, pero creen la acción igual.
- *Con `tiempo_arranque` en 0 el personaje hace cosas raras:* se está dividiendo por cero. Usen como mínimo 0.05.
- *En 2.5D salta más bajo de lo que dice la cuenta:* quedó `get_gravity()` en vez de `gravedad`, o la cuenta usa la gravedad de Godot (9.8) y no la de su juego.

Con los números de su juego (nivel 2). Receta A sin correr. Propongan la rapidez (más rápida que el perseguidor, que va a 80 px/s = 2.5 m/s) y el arranque.

Cómo saber que funciona con sus números. Con 32 px = 1 m, el perseguidor va a 2.5 m/s. Si el héroe va a 3.5 m/s, le saca 1 m por segundo.

Commit. `T7 N2 - Tarea A - arranque y frenado`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

Nivel 3 — Estados del movimiento, con un número cada uno

Qué ya saben. El `enum` y el `match` de la receta B y de la Sesión 5. Lo nuevo es una **tabla con un número por estado**: un arreglo `const` donde la posición de cada número es la del estado en el `enum`. `RUIDO[Estado.CORRER]` es el ruido al correr.

El ejemplo: los estados QUIETO, CAMINAR, CORRER y AIRE, y el ruido que hace cada uno: 0, 2, 6 y 0 m (a esa distancia lo oye un enemigo). Cada juego tiene su propio número por estado: el recuadro de su juego dice cuál (ruido, consumo de energía, visibilidad, rapidez máxima…).

Antes de programar

1. Llenen la tabla: para cada fila, ¿qué estado y qué ruido le tocan?

¿En el suelo?	Rapidez	Estado	Ruido
sí	0		
sí	2.5		
sí	4.17		
no	3		

2. Con el arranque de la receta A, al empezar a correr la rapidez pasa por 2.83, 3.17, 3.5… ¿En qué cuadro de *Avanzar y la acción desde el cuadro 30* pasa a CORRER? ¿Por qué el límite es `speed_caminar + 0.1` y no `speed_correr`?
3. Para su juego: ¿qué estados necesita su personaje, y qué número tiene cada uno? Salen del recuadro de su juego. Si alguno falta en el documento, propónganlo.

Guía para la 1. El orden importa: primero se pregunta si está en el aire, porque en el aire da igual qué tan rápido va.

Guía para la 2. Con otros números: si camina a 3 y corre a 6, con el límite en 6 solo cuenta como CORRER al llegar justo a 6. Mientras acelera de 3 a 6, seguiría marcando CAMINAR aunque ya va corriendo.

Guía para la 3. Un estado es algo que el jugador **ve o siente distinto**: si dos estados tienen el mismo número y la misma animación, sobra uno.

Paso a paso

1. Arriba de todo, debajo de `extends CharacterBody3D`, escriban el `enum` y la tabla `const` (esqueleto 1). Debajo de los enchufes, `estado` y la variable de su número.
2. **Borren** el bloque de animaciones (los `anim.play`). Dejen solo el `if direction.length() > 0:` con el `basis`.
3. Después de `move_and_slide()`, y **antes** de `_prueba()`, escriban las líneas del esqueleto 2.
4. Escriban `elegir_estado()` y `animar()` debajo de `_physics_process()`, antes del bloque de pruebas.
5. Prueba: *Avanzar y la acción desde el cuadro 30* con *correr*. Comparen con su pregunta 2.
6. **Con sus números.** Cambien los estados y la tabla por los de su juego. Si su juego tiene un estado que no viene de la rapidez (agacharse, sigilo), pregunten por su tecla en `elegir_estado()`, **antes** de las preguntas por la rapidez.

Esqueleto 1 (arriba del script):

```
extends CharacterBody3D

enum Estado { QUIETO, CAMINAR, CORRER, ____ }
const RUIDO = [0.0, 2.0, ____, 0.0]   # metros, en el orden del enum

    # (debajo de los enchufes)
var estado = Estado.QUIETO
var ruido = 0.0
```

Esqueleto 2 (después de `move_and_slide()`):

```
  move_and_slide()
    var antes = estado
    estado = elegir_estado()
    ruido = RUIDO[____]
    if estado != antes:
        print("estado ", Estado.keys()[estado], " · ruido ", ruido, " m")
    animar()
    _prueba()


func elegir_estado():
    if not is_on_floor():
        return Estado.____
    var rapidez = Vector2(velocity.x, velocity.z).length()
    if rapidez < 0.1:
        return Estado.QUIETO
    if rapidez > ____ + 0.1:
        return Estado.CORRER
    return Estado.CAMINAR


func animar():
    if accion != "":
        anim.play(accion)
        return
    match estado:
        Estado.QUIETO:
            anim.play("idle")
        Estado.CAMINAR:
            anim.play("walk")
        Estado.CORRER:
            anim.play("____")
        Estado.AIRE:
            anim.play("fall")
```

En 2.5D, la rapidez es `absf(velocity.x)` y en el aire se usa *jump* o *fall* según el signo de `velocity.y`, como en la plantilla.

Qué deben ver (ejemplo). Al arrancar: "estado CAMINAR · ruido 2.0 m". Después, "cuadro **31**: Personaje.estado cambia a CORRER" y "estado CORRER · ruido 6.0 m": en el cuadro 31 la rapidez ya es 2.83, mayor que 2.5 + 0.1. Al final de la prueba, cuando suelta las teclas y frena, vuelve a "estado CAMINAR".

Pistas

- **Pista 1:** la posición en la tabla `RUIDO` es el estado: `RUIDO[estado]`.
- **Pista 2:** `Estado.keys()[estado]` da el nombre del estado como texto, para el `print`.
- **Pista 3:** si el estado va un cuadro atrasado, `elegir_estado()` se llamó antes de `move_and_slide()`.

Errores típicos

- *"Out of bounds get index":* la tabla tiene menos números que el `enum` tiene estados.
- *El print sale en todos los cuadros:* falta el `if estado != antes`.
- *Corre con la animación de caminar:* el bloque viejo de `anim.play` sigue en `_physics_process()` y pisa a `animar()`.
- *Nunca pasa a CORRER:* el límite es `speed_correr` (ver la pregunta 2), o la acción *correr* no existe.

Con los números de su juego (nivel 3). Estados QUIETO, CAMINAR y APUNTANDO: mientras se mantiene **apuntar** (*Mouse Button Right*) camina a la rapidez que propongan. El número de cada estado es la rapidez.

Commit. `T7 N3 - Tarea A - estados con su número`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. Y las filas de su tabla de números, completas. El código no va: está en el commit.

Filas de su tabla de números (agreguen las que necesiten):

Qué es	Número	¿Ya estaba o lo propuso?
Rapidez del héroe	proponer	Lo proponen (vacío)
Escala	32 px = 1 m	Lo proponen (vacío)
Rapidez apuntando	proponer	Lo proponen (vacío)

Tarea B — Gómez Valbuena Diego Alejandro: La espada

Sigue la **receta B (una acción con tiempo)**. Tiene tres niveles: cada uno termina en un commit.

En su juego. El golpe de espada con clic izquierdo.

Lo que ya dice su documento. Espada: **40 px** de alcance, **1 de daño**, recarga **0.4 s**. Cópienlo tal cual en la tabla.

El vacío que llena. Cuánto dura el golpe. Propongan el número, escriban por qué y compruébenlo con la prueba.

Cada nivel se hace **dos veces**: primero con los **números del ejemplo** de la receta, que no son de ningún equipo (si la prueba da lo mismo que *Qué deben ver*, el código está bien), y después con los **números de su juego**, que van a la tabla.

Nivel 1 — El nodo de la acción

Qué ya saben. Una acción del Input Map se pregunta de dos formas: `Input.is_action_pressed("x")` es verdadero **mientras** la tecla está presionada (así se corre), y `Input.is_action_just_pressed("x")` es verdadero **solo en el cuadro** en que se presionó. Un nodo hijo del Personaje tiene su propio script y corre su propio `_physics_process()`; con `get_parent()` llega al Personaje.

El ejemplo: *rodar*, con Ctrl.

Antes de programar

1. Con *Mantener la acción 2 s*, la tecla queda presionada 120 cuadros. Con `is_action_just_pressed`, ¿cuántas veces escribe "empieza"? ¿Y si usaran `is_action_pressed`?
2. ¿Por qué el script va en un nodo hijo y no dentro de *personaje.gd*? Piensen en quién más está editando *personaje.gd* hoy.
3. Para su juego: ¿cómo se llama su acción y con qué tecla se hace?

Guía para la 1. Con otros números: una tecla mantenida 1 s son 60 cuadros; `just_pressed` da verdadero en 1 de ellos y `pressed` en los 60.

Guía para la 2. Dos personas editando el mismo archivo en el mismo repositorio terminan pisándose. Cada tarea en su archivo.

Guía para la 3. Está en el recuadro *Con los números de su juego (nivel 2)*, más abajo.

Paso a paso

1. **La acción.** *Project Settings* → **Input Map** → *Add New Action*: **rodar** → **+ Add** → **+** → tecla **Ctrl** → **OK**.
2. **El nodo.** En *main.tscn*, clic derecho sobre **Personaje** → *Add Child Node* → **Node** (el básico, sin 2D ni 3D). Pónganle el nombre de la acción: **Rodar**.
3. **El script.** Con el nodo seleccionado, *Attach Script* → *rodar.gd*, en la carpeta principal del proyecto. Ese archivo es **suyo**: nadie más lo toca.
4. Escriban el esqueleto.
5. En el Personaje: **Prueba** = *Mantener la acción 2 s* y **Accion Prueba** = *rodar*. F5.
6. A mano: presionen Ctrl tres veces y miren el contador.

Esqueleto (*rodar.gd*):

```
extends Node

@export var tecla = "____"
@onready var pj = get_parent()

var usos = 0


func _physics_process(delta):
    if Input.is_action_just_pressed(____):
        usos += 1
        print("empieza: ", tecla, " · uso número ", usos)
```

Qué deben ver (ejemplo). Con *Mantener la acción 2 s*: **una sola línea**, "empieza: rodar · uso número 1", aunque la tecla siga presionada 120 cuadros. A mano, con tres toques: uso número 1, 2 y 3.

Pistas

- **Pista 1:** la variable `tecla` guarda el nombre de la acción; es la que se pregunta.
- **Pista 2:** si la prueba dice "falta la acción", el nombre del Input Map y el de `tecla` no son iguales.

Errores típicos

- *120 líneas en vez de una:* usaron `is_action_pressed`.
- *"Invalid call. Nonexistent function 'is_action_just_pressed' in base 'Nil'":* escribieron `input.` en minúscula; es `Input`.
- *El nodo no aparece en la prueba ni hace nada:* no es hijo del Personaje, o el script no quedó pegado (el ícono del nodo no tiene el papelito).

Con los números de su juego (nivel 1). Cambien `tecla` por el nombre de la acción de su juego (está en el recuadro *Con los números de su juego (nivel 2)*, más abajo) y créenla en el Input Map con su tecla. *Mantener la acción 2 s* tiene que escribir una sola línea.

Commit. `T7 N1 - Tarea B - el nodo de la accion`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

Nivel 2 — Una acción con tiempo

Qué ya saben. En la Sesión 5, el fantasma tenía un `enum Estado` y un `match` que decidía qué hacer en cada estado. Una acción con tiempo es igual de simple. Tiene tres estados: **LISTA** (se puede usar), **ACTIVA** (está pasando) y **RECARGA** (ya pasó y todavía no se puede repetir). La variable `tiempo` cuenta hacia atrás en segundos: en cada cuadro se le resta `delta`. `get_parent()` da el nodo padre; aquí es el Personaje, y por él se llega a sus

enchufes.

El ejemplo: *rodar* (con Ctrl). Dura 0.3 s y se recarga en 1.2 s.

Antes de programar

1. ¿Cuántos cuadros dura ACTIVA? ¿Y RECARGA?
2. Si presionan *rodar* en el cuadro 30, ¿en qué cuadro se puede volver a rodar?
3. Si presionan otra vez durante la recarga, ¿qué pasa? ¿En qué parte del `match` se pregunta por la tecla?

Guía para la 1. Segundos × 60 = cuadros. Con otros números: 0.5 s son 30 cuadros y 2 s son 120.

Guía para la 2. Con otros números: si dura 0.5 s, se recarga en 2 s y se presiona en el cuadro 10, se puede repetir en el cuadro 10 + 30 + 120 = 160.

Guía para la 3. Miren el esqueleto: ¿en cuál de los tres estados aparece `is_action_just_pressed`?

Paso a paso

1. Abran el script del nodo del nivel 1 (en el ejemplo, *rodar.gd*) y reemplacen todo su contenido por el esqueleto: el `enum`, las dos variables de tiempo y el `match`. El nodo y la acción ya existen.
2. Seleccionen el nodo y miren el Inspector: **Tecla** = *rodar*, **Duracion** = 0.3 y **Recarga** = 1.2.
3. **Prueba.** En el Personaje, **Prueba** = *Avanzar y la acción desde el cuadro 30* y **Accion Prueba** = *rodar*. F5. Después, *Mantener la acción 2 s*.
4. **Con sus números.** Cambien en el Inspector la tecla y los dos tiempos por los de su tarea, y vuelvan a probar.

Esqueleto.

```
extends Node

enum Estado { LISTA, ACTIVA, ____ }

@export var tecla = "____"
@export var duracion = ____
@export var recarga = ____
@onready var pj = get_parent()

var estado = Estado.LISTA
var tiempo = 0.0


func _physics_process(delta):
    tiempo -= ____
    match estado:
        Estado.LISTA:
            if Input.is_action_just_pressed(tecla):
                empezar()
        Estado.ACTIVA:
            if tiempo <= 0.001:   # 0.001 y no 0: restar 1/60 no da exacto
                terminar()
        Estado.RECARGA:
            if tiempo <= 0.001:
                estado = Estado.____


func empezar():
    estado = Estado.ACTIVA
    tiempo = ____
    print("empieza: ", tecla)


func terminar():
    estado = Estado.RECARGA
    tiempo = ____
    print("termina: ", tecla)
```

Qué deben ver (ejemplo). Con *Avanzar y la acción desde el cuadro 30*: "empieza: rodar", "cuadro **32**: Rodar.estado cambia a ACTIVA", "termina: rodar" y "cuadro **50**: Rodar.estado cambia a RECARGA". Son 18 cuadros, 0.3 s. Con *Mantener la acción 2 s*: ACTIVA en el cuadro 2, RECARGA en el 20 y **LISTA en el 92**, es decir, 72 cuadros (1.2 s) de recarga. Aunque la tecla sigue presionada, no vuelve a rodar: `is_action_just_pressed` solo cuenta el instante en que se presiona. La prueba presiona al final del cuadro 30 y el cambio aparece en el 32. No importa en qué

cuadro empieza: lo que se mide es la diferencia entre dos cuadros. Por ahora el personaje no hace nada distinto mientras rueda; eso es el nivel 3.

Pistas

- **Pista 1:** lo que se le resta a `tiempo` en cada cuadro es lo que dura un cuadro.
- **Pista 2:** al empezar, `tiempo` vale lo que dura la acción; al terminar, lo que dura la recarga.
- **Pista 3:** si nunca vuelve a LISTA, revisen el último caso del `match`.

Errores típicos

- *La acción se repite mientras mantienen la tecla:* usaron `is_action_pressed` en vez de `is_action_just_pressed`. (Si su tarea **es** mantener una tecla, lo dice el recuadro de su juego y se usa el nivel 3.)
- *Dura un cuadro de más (19 en vez de 18):* compararon `tiempo <= 0` en vez de `tiempo <= 0.001`.
- *Nunca vuelve a LISTA:* en `Estado.RECARGA` falta cambiar el estado, o el nombre está mal escrito.
- *La prueba no escribe el estado de su nodo:* el nodo no es hijo del Personaje, o la variable no se llama `estado`.
- *"Identifier "Estado" not declared":* el `enum` no está arriba del script, o está escrito distinto.

Con los números de su juego (nivel 2). Receta B: acción **espada** (*Mouse Button Left*), `recarga = 0.4` y la duración que propongan.

Cómo saber que funciona con sus números. Con un golpe de 0.2 s: 3 golpes al perseguidor toman 3 × 0.2 + 2 × 0.4 = 1.4 s.

Commit. `T7 N2 - Tarea B - estados con tiempo`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

Nivel 3 — La acción cambia al personaje, con su regla

Qué ya saben. Los enchufes del Personaje (`multiplicador`, `empuje`, `accion`) y la acción con tiempo del nivel 2. `pj.basis.z` es la flecha que va hacia la **espalda** del Personaje, así que `-pj.basis.z` es hacia donde mira.

El ejemplo: mientras *rodar* está ACTIVA, el Personaje sale disparado hacia adelante a 12 m/s, las teclas no lo mueven (`multiplicador = 0`) y hace la animación *crouch*. Al terminar, todo vuelve a la normalidad. La regla es: solo se puede rodar en el suelo.

Antes de programar

1. ¿Cuántos metros avanza mientras rueda?
2. ¿Por qué el `multiplicador` se pone en 0 mientras rueda? ¿A qué rapidez iría si se quedara en 1 y el

personaje caminara a 3?

3. Si el Personaje está quieto y mirando a la cámara, ¿hacia dónde rueda?
4. Para su tarea: ¿qué enchufe usa su acción, con qué valor, y cuál es la regla que dice el recuadro de su juego?

Guía para la 1. Con otros números: a 20 m/s durante 0.2 s recorre 4 m.

Guía para la 2. Los dos se suman en las líneas de `velocity`: la rapidez de las teclas por el multiplicador, más el empuje.

Guía para la 3. `-pj.basis.z` no depende de las teclas, sino de hacia dónde mira.

Guía para la 4. Con otras acciones: una poción que deja inmóvil 0.8 s usa `multiplicador = 0` y la animación *pick-up*; un golpe usa `accion = "attack-melee-right"`; agacharse usa `multiplicador = 0.5` mientras se mantiene la tecla.

Paso a paso

1. En `empezar()`, después de `tiempo = duracion`, cambien los tres enchufes (esqueleto).
2. En `terminar()`, devuélvanlos a la normalidad: empuje en cero, multiplicador en 1 y acción vacía.
3. Agreguen la función `puede()` con la regla y úsenla en el `if` de LISTA.
4. Agreguen el `@export` del número nuevo (en el ejemplo, `rapidez`).
5. Prueba: *Avanzar y la acción desde el cuadro 30* con *rodar*.
6. **Con sus números.** Cambien el enchufe, la animación, el número y la regla por los de su tarea.
7. **Si su tarea es mantener una tecla** (enfriar, reanimar, agacharse…): en LISTA usen `is_action_pressed`, y en ACTIVA pregunten si la soltó (`not Input.is_action_pressed(tecla)`): en ese caso, terminar. El recuadro de su juego dice si es así.

Esqueleto.

```
@export var rapidez = ____

        Estado.LISTA:
            if Input.is_action_just_pressed(tecla) and puede():
                empezar()


func puede():
    return pj.____()   # la regla del ejemplo: solo en el suelo


func empezar():
    estado = Estado.ACTIVA
    tiempo = duracion
    pj.empuje = -pj.basis.z * ____
    pj.multiplicador = ____
    pj.accion = "____"
    print("empieza: ", tecla)


func terminar():
    estado = Estado.RECARGA
    tiempo = recarga
    pj.empuje = Vector3.____
    pj.multiplicador = ____
    pj.accion = ""
    print("termina: ", tecla)
```

Qué deben ver (ejemplo). Con *Avanzar y la acción desde el cuadro 30*: en el cuadro 31 va en z = −1.55, caminando a 3. En el cuadro **32** cambia a ACTIVA, la rapidez es **12.0** y la animación *crouch*. En el cuadro **50** cambia a RECARGA, vuelve a *walk* y a rapidez **3.0**, y va en z = −5.2: avanzó 3.6 m rodando (12 × 0.3), más 0.05 del primer cuadro caminando. Con *Mantener la acción 2 s*, sin moverse, rueda igual hacia −Z, porque el Personaje mira hacia allá al empezar.

Pistas

- **Pista 1:** lo que se cambia en `empezar()` se deshace en `terminar()`, con los valores de la plantilla.
- **Pista 2:** `pj.empuje` es un Vector3: una dirección (de largo 1) por una rapidez.
- **Pista 3:** si su regla depende de otra variable del Personaje (por ejemplo, el `estado` de la receta A), se lee igual: `pj.estado`.

Errores típicos

- *Rueda pero sigue con walk:* falta `pj.accion`, o el nombre de la animación está mal escrito (van en

minúscula y con guion: *attack-melee-right*).

- *Al terminar se queda quieto para siempre:* `terminar()` no devuelve el multiplicador a 1.
- *Rueda hacia atrás:* usaron `pj.basis.z` sin el signo menos.
- *Rueda a 15 en vez de 12:* el multiplicador se quedó en 1.
- *No rueda nunca:* `puede()` devuelve `false` siempre. Impriman `puede()` cuando presionan la tecla.

Con los números de su juego (nivel 3). Durante el golpe, `multiplicador = 0` y `accion = "attack-melee-right"`. La regla: impriman "golpe de 1 a 1.25 m" (40 px). Si el perseguidor tiene 3 HP, ¿cuánto tarda en vencerlo?

Commit. `T7 N3 - Tarea B - enchufes y regla`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. Y las filas de su tabla de números, completas. El código no va: está en el commit.

Filas de su tabla de números (agreguen las que necesiten):

Qué es	Número	¿Ya estaba o lo propuso?
Alcance	40 px = 1.25 m	Ya estaba en el documento
Daño	1	Ya estaba en el documento
Recarga	0.4 s	Ya estaba en el documento
Duración del golpe	proponer	Lo proponen (vacío)

Tarea C — Mejía Poveda Bryan Smith: El perseguidor

Sigue la **receta C (otro personaje que se mueve solo)**. Tiene tres niveles: cada uno termina en un commit.

En su juego. El enemigo que persigue al héroe por la sala.

Lo que ya dice su documento. Perseguidor: **3 HP**, **80 px/s**, quita 1 corazón. Cópienlo tal cual en la tabla.

El vacío que llena. Cuántos enemigos hay por sala y a qué distancia ven al héroe. Propongan el número, escriban por qué y compruébenlo con la prueba.

Cada nivel se hace **dos veces**: primero con los **números del ejemplo** de la receta, que no son de ningún equipo (si la prueba da lo mismo que *Qué deben ver*, el código está bien), y después con los **números de su juego**, que van a la tabla.

Nivel 1 — La escena del enemigo

Qué ya saben. El Personaje es un CharacterBody3D con una cápsula y un Modelo. Un enemigo, un civil o un rival

es **otro** CharacterBody3D igual, en su propia escena, que se **instancia** en *main.tscn* (se arrastra el *.tscn* a la escena). Como el Personaje, necesita la gravedad y `move_and_slide()` para apoyarse en el suelo. La resta `objetivo.global_position - global_position` es la flecha que va del enemigo al objetivo, y su largo (`length()`) es la distancia. Con `@export var objetivo: Node3D`, el objetivo se elige en el Inspector.

El ejemplo: un **Guardia** en (−6, 0, 6), quieto, que mide cada medio segundo a qué distancia está el Personaje.

Antes de programar

1. El Guardia está en (−6, 0, 6) y el Personaje en (0, 0, 0). ¿A qué distancia están, medida en el suelo?
2. Con *Avanzar 60 cuadros*, el Personaje termina en (0, 0, −3). ¿A qué distancia queda del Guardia?
3. ¿Por qué se pone `hacia.y = 0.0` antes de medir? Piensen en un objetivo que está saltando.

Guía para la 1. La distancia en el suelo es $\sqrt{(x^2 + z^2)}$. Con otros números: de (3, 0, −4) a (0, 0, 0) hay $\sqrt{(9 + 16)} = 5$ m.

Guía para la 2. La misma cuenta, con la diferencia en z. Con otros números: de (−6, 0, 6) a (0, 0, 2) hay $\sqrt{(36 + 16)} = 7.21$ m.

Guía para la 3. Si el objetivo salta, la flecha apunta hacia arriba y la distancia crece aunque esté encima.

Paso a paso

1. **La escena del enemigo.** *Scene* → *New Scene* → *Other Node* → **CharacterBody3D**. Nómbrenlo como su enemigo o NPC (en el ejemplo, **Guardia**).
2. Agréguenle un **CollisionShape3D** con *CapsuleShape3D* (radio 0.35, altura 1.75, Position Y 0.875), igual que el Personaje.
3. Arrastren otro modelo de la carpeta *personaje/* (por ejemplo *character-male-d.glb*) como hijo. Renómbrenlo **Modelo** y pónganle **Rotation Y = 180** y **Scale 2.6**.
4. *Attach Script* al Guardia, con el nombre de su enemigo (*guardia.gd*). Escriban el esqueleto. Guarden la escena (*guardia.tscn*). La escena y el script son **suyos**: nadie más los toca.
5. **En *main.tscn*:** arrastren *guardia.tscn* desde el panel FileSystem hasta la escena. En su Inspector, **Objetivo** = Personaje (arrastren el nodo) y **Position** = (−6, 0, 6).
6. **En 2.5D:** en el Inspector del Guardia, **Axis Lock** → **Linear Z** marcado, y Position con z = 0 (por ejemplo, (−5, 0, 0); en x = −6 chocaría con la Plataforma2).
7. F5 sin tocar nada y miren el Output. Después, **Prueba** = *Avanzar 60 cuadros* en el Personaje.

Esqueleto (*guardia.gd*):

```
extends CharacterBody3D

@export var objetivo: Node3D
@onready var anim = $Modelo/AnimationPlayer


func _physics_process(delta):
    velocity += get_gravity() * delta
    anim.play("____")
    move_and_slide()
    var hacia = ____.global_position - global_position
    hacia.y = 0.0
    if Engine.get_physics_frames() % 30 == 0:
        print("distancia ", snappedf(hacia.____(), 0.01))
```

Qué deben ver (ejemplo). Quieto: "**distancia 8.49**" cada medio segundo, siempre igual. Con *Avanzar 60 cuadros*: 9.57, 10.78 y después **10.82** fijo, cuando el Personaje se detiene en (0, 0, −3). El Guardia se queda en su sitio, apoyado en el suelo y en *idle*. En 2.5D, desde (−5, 0, 0): 5.0 quieto; con *Avanzar 60 cuadros* a 4 m/s, sube hasta 9.0.

Pistas

- **Pista 1:** la flecha va **del** guardia **al** objetivo: primero la posición del objetivo y después la propia.
- **Pista 2:** si no escribe nada, impriman `objetivo` en `_ready()`. Si sale `<null>`, falta elegirlo en el Inspector de la instancia, en *main.tscn* (no en *guardia.tscn*).

Errores típicos

- *"Invalid access to property or key 'global_position' on a base object of type 'Nil'":* el Objetivo está vacío en el Inspector.
- *Se cae al vacío o se hunde:* falta el CollisionShape3D, o lo pusieron debajo del suelo (Position Y negativa).
- *Está de espaldas o mira hacia la cámara:* al Modelo le falta Rotation Y = 180.
- *"Node not found: "Modelo/AnimationPlayer"":* el modelo no se llama Modelo.
- *Es diminuto:* falta Scale 2.6.

Con los números de su juego (nivel 1). Pónganle a la escena y al script el nombre de su enemigo o NPC (recuadro *Con los números de su juego (nivel 2)*, más abajo), cámbienle el modelo si quieren, y colóquenlo donde empieza en su juego. Anoten la distancia inicial.

Commit. `T7 N1 - Tarea C - escena del enemigo`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. Y una captura del enemigo en la escena. El código no va: está en el commit.

Nivel 2 — Otro personaje que se mueve solo

Qué ya saben. Del nivel 1: el enemigo es otro CharacterBody3D con su gravedad y su `move_and_slide()`, y la flecha `objetivo.global_position - global_position`, cuyo largo (`length()`) es la distancia. Lo nuevo es que esa flecha lo **mueve**: `normalized()` la deja en largo 1, y multiplicada por la rapidez da la velocidad hacia el objetivo.

El ejemplo: un guardia que camina a 2 m/s hacia el Personaje. Sale de (−6, 0, 6) y el Personaje está quieto en (0, 0, 0). Se detiene a 1 m.

Antes de programar

1. ¿A qué distancia del Personaje empieza el guardia?
2. ¿Cuánto tarda en llegar a 1 m? ¿Cuántos cuadros son? ¿Cuánto baja la distancia cada 30 cuadros?
3. ¿Qué pasaría si no se detuviera a 1 m? Piensen qué dirección da `normalized()` cuando la distancia es casi 0.

Guía para la 1. La distancia en el suelo es $\sqrt{(x^2 + z^2)}$. Con otros números: de (3, 0, −4) a (0, 0, 0) hay $\sqrt{(9 + 16)} = 5$ m.

Guía para la 2. Con otros números: si está a 5 m, se detiene a 1 m y camina a 4 m/s, tarda (5 − 1) ÷ 4 = 1 s, 60 cuadros. Cada 30 cuadros baja 4 × 0.5 = 2 m.

Guía para la 3. Si están casi encima, la flecha es diminuta y en cada cuadro apunta para un lado distinto.

Paso a paso

1. Abran *guardia.gd* (el script del nivel 1) y reemplacen su contenido por el esqueleto. La escena, el Modelo y el Objetivo ya quedaron en el nivel 1.
2. Dejen al Guardia donde quedó en el nivel 1: (−6, 0, 6) (en 2.5D, (−5, 0, 0)).
3. F5 sin tocar el teclado y miren el Output.
4. **Con sus números.** Cambien la rapidez (y la distancia a la que se detiene, si su tarea la pide) por las de su juego, y vuelvan a probar.

Esqueleto.

```
extends CharacterBody3D

@export var objetivo: Node3D
@export var rapidez = ____
@onready var anim = $Modelo/AnimationPlayer


func _physics_process(delta):
    velocity += get_gravity() * delta
    var hacia = ____.global_position - global_position
    hacia.y = 0.0
    var distancia = hacia.____()
    if distancia > ____:
        var dir = hacia.normalized()
        velocity.x = dir.x * ____
        velocity.z = dir.z * rapidez
        basis = Basis.looking_at(dir)
        anim.play("walk")
    else:
        velocity.x = 0.0
        velocity.z = 0.0
        anim.play("____")
    move_and_slide()
    if Engine.get_physics_frames() % 30 == 0:
        print("distancia ", snappedf(distancia, 0.01))
```

Qué deben ver (ejemplo). Cada medio segundo, una línea: **distancia 7.52**, 6.52, 5.52, 4.52, 3.52, 2.52, 1.52 y, desde ahí, **0.99** fijo: se detuvo a 1 m y quedó en *idle*. Baja 1 m cada 30 cuadros (2 m/s × 0.5 s) y la cuenta de la pregunta 2 da 3.74 s. El guardia mira siempre hacia el Personaje. En 2.5D, se queda en z = 0.

Pistas

- **Pista 1:** la flecha va **del** guardia **al** objetivo: primero la posición del objetivo y después la propia.
- **Pista 2:** `hacia.y = 0.0` evita que intente caminar hacia arriba o hacia abajo cuando el objetivo salta.
- **Pista 3:** si no se mueve, impriman `objetivo` en `_ready()`. Si sale `<null>`, falta elegirlo en el Inspector de la instancia, en *main.tscn* (no en *guardia.tscn*).

Errores típicos

- *"Invalid access to property or key 'global_position' on a base object of type 'Nil'":* el Objetivo está vacío en el Inspector.
- *Camina de espaldas:* al Modelo le falta Rotation Y = 180.
- *Se cae al vacío o se hunde:* falta el CollisionShape3D, o lo pusieron debajo del suelo (Position Y negativa).
- *Vibra o empuja al Personaje al llegar:* falta el `if distancia > 1.0`.

- *En 2.5D se sale del plano y se va hacia la cámara:* falta Axis Lock Linear Z, o su z no es 0.
- *Se queda trabado contra una caja:* va en línea recta y no esquiva; está bien por ahora. Muévanlo a otro lado para probar.

Con los números de su juego (nivel 2). Receta C con `rapidez = 2.5` (80 px/s ÷ 32).

Cómo saber que funciona con sus números. A 2.5 m/s, la distancia baja 1.25 m cada 30 cuadros.

Commit. `T7 N2 - Tarea C - sigue al jugador`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. El código no va: está en el commit.

Nivel 3 — Patrullar, ver y perseguir

Qué ya saben. El seguidor del nivel 2 y los estados con `match`. Lo nuevo es la **histéresis**, que ya vieron en la Sesión 5: se empieza a perseguir a una distancia y se deja de perseguir a otra **más grande**. Si fueran la misma, en el borde cambiaría de estado en cada cuadro.

El ejemplo: el guardia patrulla entre (−6, 0, −3) y (6, 0, −3) a 2 m/s. Si el Personaje está a menos de 6 m, lo persigue; si se aleja a más de 9 m, vuelve a patrullar. Empieza en (−6, 0, −3) y el Personaje está quieto en (0, 0, 0).

Antes de programar

1. La patrulla va por la línea z = −3. ¿En qué x de esa línea queda el guardia a 6 m del Personaje? ¿Alcanza a llegar a x = 0 sin verlo?
2. Si el Personaje huye a 5 m/s y el guardia lo sigue a 2 m/s, ¿cuánto tarda en pasar de 6 m a 9 m de distancia?
3. ¿Qué pasaría con un solo radio de 6 m si el Personaje se queda justo a 6 m?
4. Para su juego: ¿a qué distancia ve su enemigo y a cuál lo pierde? ¿Por dónde patrulla? (Sale del recuadro de su juego; si falta, propónganlo.)

Guía para la 1. Distancia² = x² + z². Con otros números: en la línea z = −4, a 5 m del origen, x² = 25 − 16 = 9, así que x = −3 o x = 3.

Guía para la 2. La distancia crece a la diferencia de las dos rapideces. Con otros números: 6 contra 4 m/s, se separan a 2 m/s.

Guía para la 3. Piensen en un guardia que da un paso hacia adelante (5.99 m: lo ve) y otro hacia atrás (6.01 m: lo pierde).

Paso a paso

1. Agreguen el `enum`, los dos radios y la lista de puntos de la patrulla (esqueleto). La lista se puede cambiar en el Inspector.

2. Cambien `_physics_process()` por el `match` del esqueleto, y escriban la función `ir_a(p)`, que es el movimiento del nivel 2 hacia cualquier punto.
3. Coloquen el guardia en (−6, 0, −3). F5 sin tocar nada: tiene que escribir "lo vio a …".
4. A mano: corran lejos del guardia. Cuando pase de 9 m tiene que escribir "lo perdió a …" y volver a patrullar.
5. **Con sus números.** Cambien los radios, la rapidez y los puntos por los de su juego, y vuelvan a probar. En 2.5D, los puntos tienen z = 0.

Esqueleto.

```
extends CharacterBody3D

enum Estado { PATRULLA, PERSEGUIR }

@export var objetivo: Node3D
@export var rapidez = 2.0
@export var radio_ver = ____
@export var radio_perder = ____
@export var puntos: Array[Vector3] = [
        Vector3(-6, 0, -3), Vector3(6, 0, -3)]
@onready var anim = $Modelo/AnimationPlayer

var estado = Estado.PATRULLA
var punto = 0


func _physics_process(delta):
    velocity += get_gravity() * delta
    var distancia = global_position.distance_to(objetivo.global_position)
    match estado:
        Estado.PATRULLA:
            if distancia < ____:
                estado = Estado.PERSEGUIR
                print("lo vio a ", snappedf(distancia, 0.01), " m")
            ir_a(puntos[punto])
            if global_position.distance_to(puntos[punto]) < 0.3:
                punto = (punto + 1) % puntos.size()
        Estado.PERSEGUIR:
            if distancia > ____:
                estado = Estado.____
                print("lo perdió a ", snappedf(distancia, 0.01), " m")
            ir_a(objetivo.global_position)
    move_and_slide()


func ir_a(p):
    var hacia = p - global_position
    hacia.y = 0.0
    if hacia.length() < 0.05:
        return
    var dir = hacia.normalized()
    velocity.x = dir.x * rapidez
    velocity.z = dir.z * rapidez
```

```
    basis = Basis.looking_at(dir)
    anim.play("walk")
```

Qué deben ver (ejemplo). En el primer medio segundo: "**lo vio a 5.97 m**". Lo ve casi al arrancar, porque desde x = −5.2 ya está a menos de 6 m. Va hacia el Personaje y se queda pegado a él. Si corren lejos, al pasar de 9 m escribe "lo perdió a" y un número apenas mayor que 9, y vuelve hacia su punto de patrulla. Si mueven al Personaje de golpe a (0, 0, 12), escribe "**lo perdió a 12.4 m**".

Pistas

- **Pista 1:** `radio_perder` tiene que ser más grande que `radio_ver`.
- **Pista 2:** `(punto + 1) % puntos.size()` da la vuelta: después del último punto vuelve al primero.
- **Pista 3:** si nunca lo pierde, revisen que en PERSEGUIR se pregunte por `radio_perder` y no por `radio_ver`.

Errores típicos

- *Cambia entre PATRULLA y PERSEGUIR en cada cuadro:* los dos radios son iguales.
- *Se queda quieto en el primer punto:* el umbral de "llegué" (0.3) es más chico que lo que alcanza a acercarse, o el punto está dentro de una caja.
- *Patrulla flotando o se hunde:* la `y` de los puntos no es 0 (el suelo está en y = 0).
- *"Invalid get index '1' (on base: 'Array')":* la lista `puntos` tiene un solo punto; necesita al menos dos.

Con los números de su juego (nivel 3). PATRULLA y PERSEGUIR con los radios que propongan (en una sala sellada, el `radio_ver` puede ser el tamaño de la sala).

Commit. `T7 N3 - Tarea C - patrulla y persecucion`, y *Push origin*.

Qué va al documento. Las respuestas de *Antes de programar*, lo que escribió el Output con el ejemplo y con sus números (copiado o en captura), y una línea sobre si su predicción coincidió y, si no, por qué. Y las filas de su tabla de números, completas. El código no va: está en el commit.

Filas de su tabla de números (agreguen las que necesiten):

Qué es	Número	¿Ya estaba o lo propuso?
Rapidez del perseguidor	80 px/s = 2.5 m/s	Ya estaba en el documento
Radios	proponer	Lo proponen (vacío)
Enemigos por sala	proponer	Lo proponen (vacío)

Extra — Para quien termine todo

1. **La tarea de al lado.** El nivel 1 y el 2 de otra tarea de esta guía, en un archivo aparte, sin tocar el de su

compañero. Con su commit: `T7 N1 - Tarea X - extra`.

2. **Que los pies no patinen.** Usen `anim.speed_scale` para que la animación de caminar vaya proporcional a la rapidez real (por ejemplo, `avance.length() ÷ speed_caminar`), y vuélvanlo a 1 en los otros estados.

Documento de entrega

Un documento por estudiante (PDF o enlace), en su fila de la hoja del taller en el Excel del corte. Trae, en este orden:

1. **Nombre completo**, **usuario de GitHub** (el que aparece en sus commits) y la **URL del repositorio** del equipo.
2. **Su tarea** (A, B o C, con su receta) y el nombre de su archivo.
3. **Sus commits:** por cada nivel, el mensaje y el enlace al commit (en GitHub, *Commits* → el commit → copiar la URL).
4. **Por cada nivel:** lo que pide **Qué va al documento**. El código **no** se copia: está en el commit.
5. **La tabla de números del juego**, completa.
6. Respuestas cortas, con sus palabras:
7. **a.** ¿Por qué la entrada del teclado se gira con la cámara y no se usa tal cual en los ejes del mundo?
8. **b.** ¿Por qué la diagonal salía torcida con `camara.global_basis` y derecha con `rotated(Vector3.UP, ...)`?
9. **c.** ¿Qué vacío del documento llenó su tarea? ¿Qué número propuso y por qué ese y no otro?
10. **d.** Cuando lo probó en el juego, ¿el número se sintió bien? Usen la medida de la prueba para decir si lo dejaría así o lo cambiaría, y a cuánto.
11. **Anexo IA:** si usaron IA, copien cada prompt que escribieron y la respuesta completa que obtuvieron. Si no la usaron, escriban "No se usó IA". Si usaron IA y no hay anexo, el taller no se califica.

Cómo se califica (0.0 a 5.0)

Cada nivel se califica en **su commit** (el código corre y da lo que dice *Qué deben ver*, con el ejemplo y con sus números) más lo del documento. Un nivel sin commit con su usuario se califica solo por el documento, y vale menos. Cada franja exige todo lo de la anterior; si una franja queda a medias, la nota queda entre las dos (por ejemplo, 3.5 o 4.5).

Nota	Grupos de 3 horas	Grupos de 2 horas
3.0	Nivel 1 de su tarea: el commit, las pruebas con el ejemplo comparadas con *Qué deben ver* y con sus números, y las respuestas de *Antes de programar*.	Nivel 1 de su tarea: el commit y las pruebas con el ejemplo.
4.0	Nivel 2 de su tarea: el commit, las pruebas con el ejemplo y con sus números, y la tabla de números con el vacío que llenó y su porqué. Respuestas a, b y c.	Nivel 2 de su tarea: el commit, la prueba con el ejemplo y con su número, y la tabla con el vacío que llenó. Respuestas a y c.
5.0	Nivel 3 de su tarea: el commit, la medida comparada con su predicción y la tabla completa. Respuesta d.	Nivel 3 de su tarea: el commit y la tabla, aunque falte una de las pruebas. Respuesta d.

Hacer bien lo pedido hasta el nivel 2 da 4.0. El 5.0 es para quien resuelve y explica el nivel 3 de su tarea. El documento sin usuario de GitHub o sin la URL del repositorio no se califica hasta que los tenga.